

MediLink

System & Data Flow Architecture

A comprehensive reference covering the full-stack design, Docker service topology, database schema, HttpOnly cookie JWT authentication flow, REST API surface, RBAC model, and Celery async pipeline of the MediLink Hospital Management System.

PROJECT

MediLink
HMS

STACK

Django · React · PostgreSQL ·
Redis

AUTH MODEL

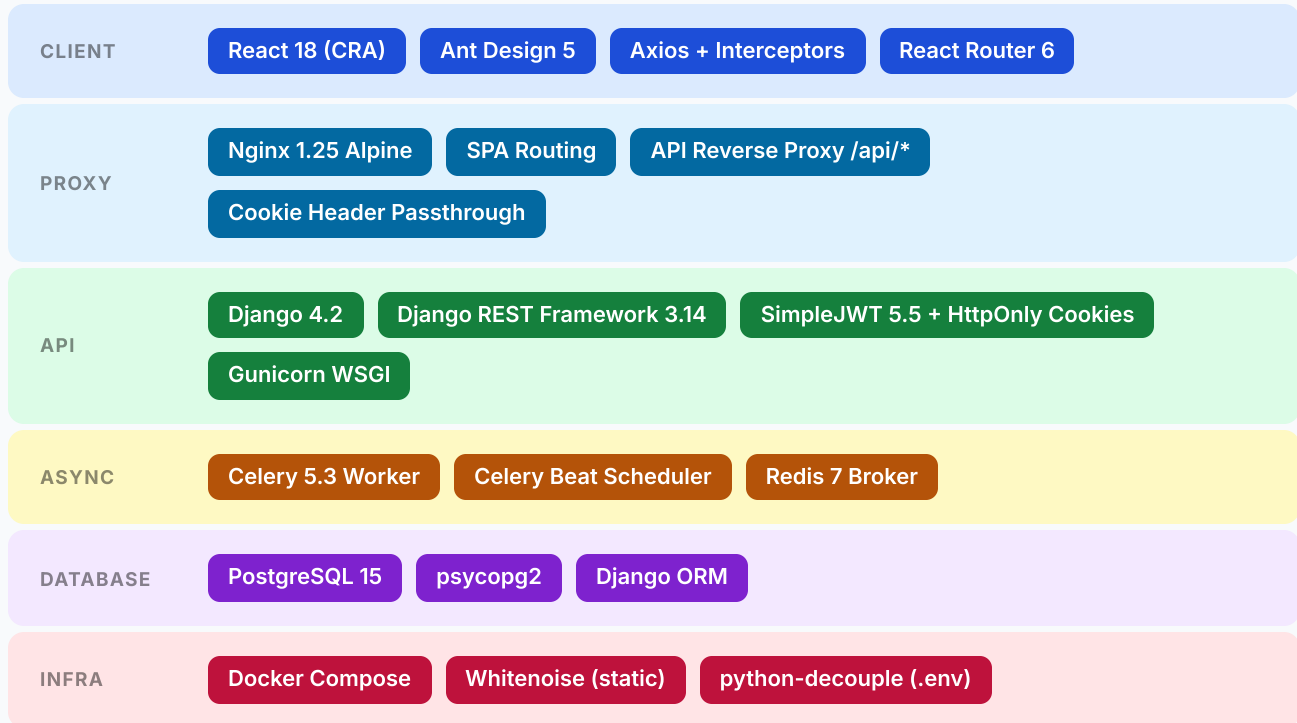
HttpOnly Cookie
JWT

DATE

September
2026

1 • Technology Stack

MediLink is a role-based hospital management system built with a decoupled architecture. A React SPA communicates with a Django REST API via JSON HTTP. All services are containerised with Docker Compose on a private bridge network.



2 · Docker Service Topology

Six containers on a private `medilink_network` bridge network. Only **frontend :3000** and **backend :8000** expose host ports. All other services communicate exclusively over the internal network.

frontend

nginx:alpine

3000:80

Serves React build (SPA)

Proxies `/api/*` to backend:8000

Forwards Set-Cookie headers

proxy_cookie_path rewriting

backend

python:3.11-slim

8000:8000

Django + Gunicorn 4 workers

Runs migrate on startup

Collects static to `/staticfiles`

Depends on: db, redis

db (PostgreSQL 15)

postgres:15-alpine

Internal only, no host port

Volume: `postgres_data`

Health-checked (`pg_isready`)

redis

redis:7-alpine

Internal only

Volume: `redis_data`

Celery broker + result backend

celery-worker

Same image as backend

cmd: `celery -A medilink worker`

Processes async task queue

Shares backend code

celery-beat

Same image as backend

cmd: `celery -A medilink beat`

Fires periodic schedules

Checks overdue invoices / hr

3 · Authentication & Token Flow

Tokens are **never exposed to JavaScript**. Django sets them as HttpOnly cookies; the browser transmits them automatically. CSRF protection uses Django's double-submit cookie pattern with X-CSRFToken header.

Login → Establish Session

- Browser** → POST /api/users/login/ { username, password }
- Django** → authenticate() → RefreshToken.for_user(user) → _set_auth_cookies(response, refresh_token)
- Set-Cookie** → medilink_access=JWT; HttpOnly; SameSite=Lax; Max-Age=3600; Path=/
- Set-Cookie** → medilink_refresh=JWT; HttpOnly; SameSite=Lax; Max-Age=604800; Path=/api/users/token/refresh/
- Response** → { user: { id, username, role, first_name, last_name } } << NO tokens in body >>

Silent Token Refresh (401 Interceptor)

- Axios** → Catches 401 → queues concurrent requests → POST /api/users/token/refresh/ (browser sends medilink_refresh cookie)
- Django** → Validates refresh JWT → blacklists old token → issues new access cookie + rotated refresh cookie
- Axios** → Replays original request; all queued requests resume automatically

Logout

- Browser** → POST /api/users/logout/ → Django blacklists refresh token in token_blacklist DB table
- Set-Cookie** → medilink_access=; Max-Age=0 medilink_refresh=; Max-Age=0 (cookies cleared)

Security properties: HttpOnly → XSS immune · SameSite=Lax → CSRF safe · Secure=true (production, HTTPS only) · Token blacklist → server-side invalidation on logout

4 · API Endpoint Reference

Auth & Users — /api/users/

METHOD	ENDPOINT	AUTH	DESCRIPTION
POST	/api/users/login/	Any	Login — sets HttpOnly JWT access + refresh cookies
POST	/api/users/logout/	JWT Cookie	Logout — blacklists refresh token, clears cookies
POST	/api/users/token/refresh/	Cookie	Silent token refresh via HttpOnly refresh cookie
POST	/api/users/	Any	Register new user (patient or doctor)
GET	/api/users/me/	JWT Cookie	Get current user profile — used for session restoration
PATCH	/api/users/{id}/	Owner/Admin	Update user profile fields
GET	/api/users/doctors/	JWT Cookie	List all doctors with DoctorProfiles
GET	/api/users/patients/	Doctor/Admin	List all patients
GET	/api/csrf/	Any	Prime CSRF cookie for SPA before first POST

Appointments — /api/appointments/

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/appointments/	JWT Cookie	List appointments (filtered by role: own for patient/doctor)
POST	/api/appointments/	JWT Cookie	Create appointment; validates (doctor, date, time) uniqueness
GET	/api/appointments/{id}/	JWT Cookie	Get appointment detail
PATCH	/api/appointments/{id}/	Doctor/Admin	Update status, notes, confirm or complete
GET	/api/appointments/schedules/	JWT Cookie	Doctor Schedule objects (availability blocks)

Medical Records — /api/medical-records/

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/medical-records/	JWT Cookie	Patient sees own; doctor sees records they created; admin sees all
POST	/api/medical-records/	Doctor	Create medical record with nested prescriptions + lab tests
GET	/api/medical-records/prescriptions/	JWT Cookie	List prescriptions (scoped by role)

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/medical-records/lab-tests/	JWT Cookie	List ordered lab tests with results
GET	/api/medical-records/vital-signs/	JWT Cookie	List vital sign recordings for patient

Billing — /api/billing/

METHOD	ENDPOINT	AUTH	DESCRIPTION
GET	/api/billing/invoices/	JWT Cookie	List invoices (patient sees own; admin sees all)
POST	/api/billing/invoices/	Admin	Create invoice; auto-generates INV-XXXXXXX number
POST	/api/billing/payments/	Admin	Record payment; triggers sync_payment_status() auto-update
GET	/api/billing/insurance-claims/	JWT Cookie	List insurance claims
POST	/api/billing/insurance-claims/	JWT Cookie	Submit claim; auto-generates CLM-XXXXXXX number

5 · Database Schema (PostgreSQL ERD)

12 tables across 4 Django apps. Foreign keys enforce referential integrity at the DB level. Unique constraints prevent double-booking and duplicate profiles.

 users_user	 users_doctorprofile
id BigInt PK	id BigInt PK
username varchar unique	user_id FK user (1:1)
email / password varchar	specialization varchar
role admin doctor patient	license_number varchar unique
first_name, last_name varchar	experience_years int
phone, address varchar/text	consultation_fee decimal
date_of_birth date	

 appointments_appointment	 medical_records_medicalrecord
id BigInt PK	id BigInt PK
patient_id FK user	patient_id FK user
doctor_id FK user	doctor_id FK user
date, time, duration date/time/int	appointment_id FK appt (1:1)
status scheduled confirmed completed	diagnosis, symptoms text
reason, notes text	treatment_plan text

 billing_invoice	 billing_payment
id BigInt PK	id BigInt PK
patient_id FK user	invoice_id FK invoice
appointment_id FK appt (nullable)	amount decimal
invoice_number INV-XXXXXXX unique	payment_method cash card insurance online
status pending paid overdue	transaction_id varchar
subtotal/tax/discount/total decimal	

Entity Relationships

User (patient)	→	Many Appointments as patient_appointments
User (doctor)	→	Many Appointments as doctor_appointments + Doctor Schedule slots
Appointment	→	One MedicalRecord (OneToOne) + One Invoice (nullable FK)
MedicalRecord	→	Many Prescriptions, LabTests; Patient has separate VitalSigns
Invoice	→	Many InvoiceItems, Payments, InsuranceClaims
User	→	One DoctorProfile OR one PatientProfile (1:1, determined by role)

6 · End-to-End Data Flows

6.1 · Appointment Booking Flow

1 Patient browses doctors & availability

`GET /api/users/doctors/` + `GET /api/appointments/schedules/` populate doctor list and available time slots in the React Appointments component.

2 Patient submits booking

`POST /api/appointments/` body: {patient, doctor, appointment_date, appointment_time, reason}. JWT cookie authenticates the request automatically via `CookieJWTAuthentication`.

3 DRF validates & persists

`AppointmentSerializer` validates `unique_together` constraint (doctor, date, time). Django ORM writes row to `appointments_appointment` table. Returns 201 Created with appointment data.

4 Doctor confirms & completes

`PATCH /api/appointments/{id}/` transitions: scheduled → confirmed → completed. Doctor then creates a `MedicalRecord` linked to this appointment (OneToOne).

6.2 · Medical Record & Billing Flow

1 Doctor creates medical record

`POST /api/medical-records/` with diagnosis, symptoms, treatment plan. Prescriptions and LabTests created as nested objects. VitalSigns recorded via separate endpoint.

2 Admin generates invoice

`POST /api/billing/invoices/` linked to appointment. Auto-generates `INV-XXXXXXX`. InvoiceItems added for consultation fee and tests. `total = subtotal + tax - discount`.

3 Patient views & pays

`GET /api/billing/invoices/` (scoped to own). `POST /api/billing/payments/` records payment. `Invoice.sync_payment_status()` auto-updates: pending → partially_paid → paid.

4 Celery Beat: overdue detection

Celery Beat fires `billing.tasks.check_overdue_invoices` every 3600s. Invoices past `due_date` with pending/partially_paid status are bulk-updated to `overdue`.

6.3 · Role-Based Access Control (RBAC)

👑 Admin

- Full CRUD on all resources
- Create & manage invoices
- Record payments for any patient
- View all appointments & records

🩺 Doctor

- View own appointments only
- Create & edit medical records
- Write prescriptions & lab orders
- Record vital signs for any patient

Access Django /admin panel

View patient list

Patient

Book appointments with any doctor

View own appointments & records

View own invoices & payments

Submit insurance claims

7 · Async Processing Pipeline (Celery + Redis)

Background tasks run outside the HTTP cycle, keeping API responses fast. Celery Beat handles scheduled jobs independently from worker tasks.

